

CS224N | Language Models and Recurrent Neural Networks

Selene

August 20, 2026

1 Language Models

A language model computes the probability that a sequence of words appears in a particular order. The probability of a sequence of m words, $\{w_1, \dots, w_m\}$, is denoted by $P(w_1, \dots, w_m)$. Because the number of words preceding w_i depends on its position in the input document, practical models usually do not condition $P(w_1, \dots, w_m)$ on the entire history. Instead, they approximate it by conditioning only on the previous $n - 1$ words:

$$P(w_1, \dots, w_m) = \prod_{i=1}^m P(w_i | w_1, \dots, w_{i-1}) \approx \prod_{i=1}^m P(w_i | w_{i-n+1}, \dots, w_{i-1}).$$

1.1 n-gram Language Models

We can estimate these probabilities by comparing the count of each n-gram with the count of its prefix. For example, the following equations show the relationship between bigram and trigram models:

$$p(w_2 | w_1) = \frac{\text{count}(w_1, w_2)}{\text{count}(w_1)},$$

$$p(w_3 | w_1, w_2) = \frac{\text{count}(w_1, w_2, w_3)}{\text{count}(w_1, w_2)}.$$

This approach allows us to predict the next word from the preceding $n - 1$ words, but how long should the context window be? In some cases, the preceding n consecutive words are not enough to capture the true context. This leads to two major problems with n-gram models: **sparsity** and **storage**.

- **Sparsity:** Consider the trigram example above. If w_1, w_2, w_3 have never appeared together in the corpus, the estimated probability of w_3 is zero. One solution is to add a small value δ to the count of every word in the vocabulary, a technique known as **smoothing**. If w_1 and w_2 have never appeared together, the conditional probability of w_3 cannot even be computed. In that case, we can condition only on w_2 , a technique known as **backoff**. Sparsity becomes increasingly severe as n grows, so in practice $n \leq 5$ is common.
- **Storage:** We must store the count of every n-gram that appears in the corpus. The required storage therefore grows as either n or the corpus size increases.

1.2 Window-based Neural Language Models

In *A Neural Probabilistic Language Model*, Bengio et al. proposed using a neural network to address the “curse of dimensionality” in language modeling. The model learns distributed word representations while using the vectors of the previous n words to predict a probability distribution over the next word.

The model above can be written as

$$\hat{y} = \text{softmax}(W^{(2)} \tanh(W^{(1)}x + b^{(1)}) + W^{(3)}x + b^{(3)}).$$

Here, $W^{(1)}$ acts on the word vectors, $W^{(2)}$ acts on the hidden layer, and $W^{(3)}$ acts directly on the word vectors.

In the simplified model above, the vectors of multiple input words are concatenated:

$$e = [e^{(1)}; e^{(2)}; e^{(3)}; e^{(4)}].$$

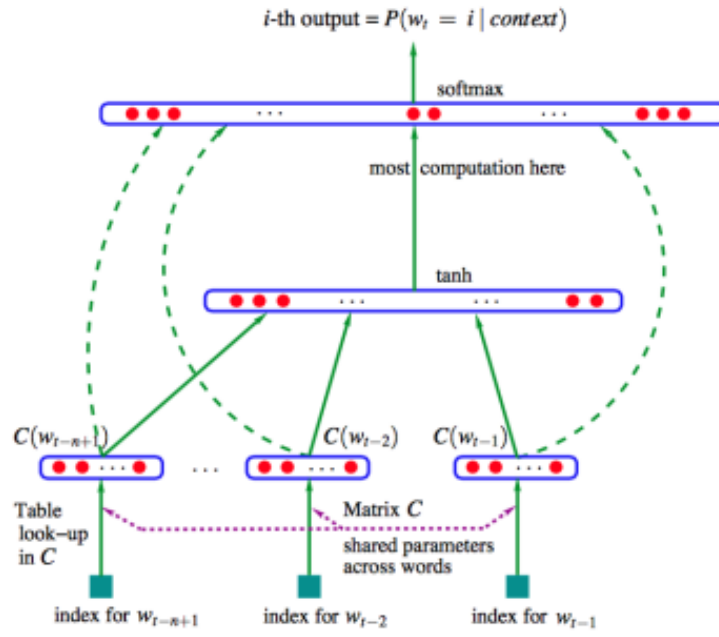


Figure 1: A window-based neural language model

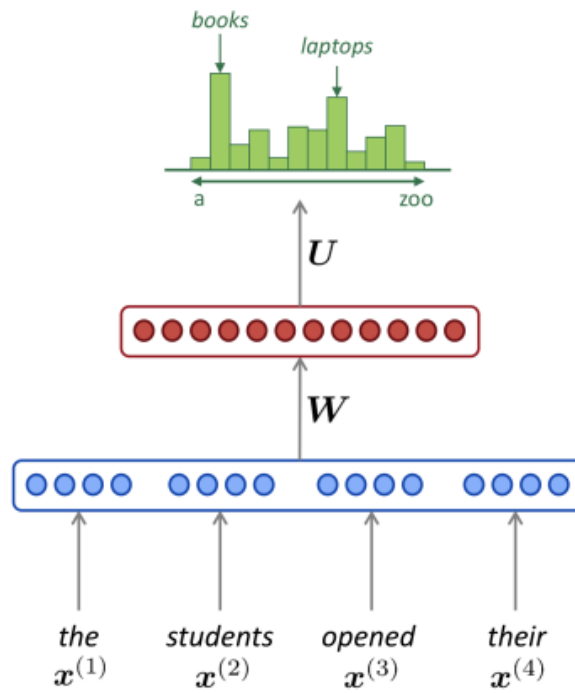


Figure 2: A simplified neural language model

The concatenated vector then passes through a hidden layer, $h = f(We + b_1)$, and finally produces the output distribution $\hat{y} = \text{softmax}(Uh + b_2)$.

2 Recurrent Neural Networks (RNNs)

Unlike traditional language models, which use only a fixed-length history window, a recurrent neural network can condition its predictions on information carried by all previous words. The following diagram shows the structure of an RNN.

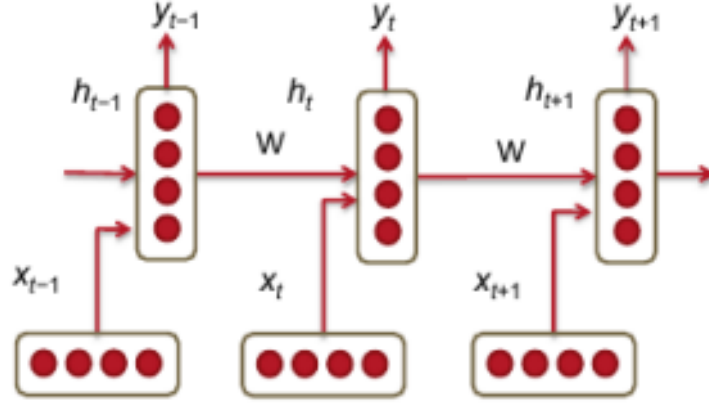


Figure 3: The structure of a recurrent neural network

Each vertical rectangle represents the hidden layer at time step t . A hidden layer contains several neurons, each of which first applies a linear matrix operation and then a nonlinear function. At each time step, the hidden layer receives two inputs: the previous hidden state h_{t-1} and the current input x_t . The former is multiplied by the weight matrix $W^{(hh)}$, while the latter is multiplied by $W^{(hx)}$. Together, they produce the hidden representation h_t . The hidden state is then multiplied by $W^{(S)}$ and passed through a softmax over the vocabulary to produce the predicted distribution $\hat{y}_t$ for the next word:

$$h_t = \sigma(W^{(hh)}h_{t-1} + W^{(hx)}x_t),$$

$$\hat{y}_t = \text{softmax}(W^{(S)}h_t).$$

The same $W^{(hh)}$ and $W^{(hx)}$ are reused at every time step. The model therefore has fewer parameters to learn; more importantly, the number of parameters is independent of the input sequence length, avoiding the curse of dimensionality.

2.1 RNN Loss and Perplexity

RNNs commonly use the cross-entropy loss introduced in the previous notes. At time step t , summing over the entire vocabulary gives

$$\mathcal{J}^{(t)}(\theta) = - \sum_{j=1}^{|V|} y_{t,j} \log \hat{y}_{t,j}.$$

For a corpus of size T , the overall cross-entropy is

$$J = \frac{1}{T} \sum_{t=1}^T \mathcal{J}^{(t)}(\theta) = - \frac{1}{T} \sum_{t=1}^T \sum_{j=1}^{|V|} y_{t,j} \log \hat{y}_{t,j}.$$

The following equation defines **perplexity**. When the logarithm in the cross-entropy is base 2, perplexity is the exponential form of that cross-entropy:

$$\text{Perplexity} = 2^J.$$

Lower perplexity means that the model is more confident in its predictions of the next word and, in general, agrees more closely with the observed sequence.

2.2 Limitations of RNNs

RNNs are not a universal solution. They have several limitations:

- Computation is slow because it proceeds sequentially and is difficult to parallelize.
- In practice, it is difficult to use information from the distant past effectively, partly because of vanishing and exploding gradients.

2.3 Vanishing and Exploding Gradients

An RNN repeatedly propagates the same weight matrix from one time step to the next. One of its goals is to carry contextual information across long temporal distances. However, information from much earlier time steps may disappear during backpropagation; this is known as the **vanishing-gradient problem**. The mathematical reason is described below.

To compute the total error gradient with respect to the parameter W , we sum the error gradients from all time steps:

$$\frac{\partial E}{\partial W} = \sum_{t=1}^T \frac{\partial E_t}{\partial W}.$$

Applying the chain rule to the gradient at a single time step gives

$$\frac{\partial E_t}{\partial W} = \sum_{k=1}^t \frac{\partial E_t}{\partial y_t} \frac{\partial y_t}{\partial h_t} \frac{\partial h_t}{\partial h_k} \frac{\partial h_k}{\partial W}.$$

Here, $\frac{\partial h_t}{\partial h_k}$ is the gradient of the hidden state at time t with respect to the hidden state at time k . By the chain rule,

$$\frac{\partial h_t}{\partial h_k} = \prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} = \prod_{j=k+1}^t W^\top \text{diag}[f'(h_{j-1})].$$

The total gradient is therefore

$$\frac{\partial E}{\partial W} = \sum_{t=1}^T \sum_{k=1}^t \frac{\partial E_t}{\partial y_t} \frac{\partial y_t}{\partial h_t} \left(\prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_k}{\partial W}.$$

Suppose that

$$\|W^\top\| \leq \beta_W, \quad \|\text{diag}[f'(h_{j-1})]\| \leq \beta_h.$$

Then

$$\left\| \frac{\partial h_j}{\partial h_{j-1}} \right\| \leq \beta_W \beta_h, \quad \left\| \frac{\partial h_t}{\partial h_k} \right\| \leq (\beta_W \beta_h)^{t-k}.$$

Therefore:

- If $\beta_W \beta_h < 1$, the gradient decays exponentially over time, producing a **vanishing gradient**.
- If $\beta_W \beta_h > 1$, the gradient may grow exponentially, producing an **exploding gradient**.

2.4 Remedies for Exploding and Vanishing Gradients

Exploding gradients

When the gradient norm exceeds a threshold, we scale the gradient to a fixed range:

$$\hat{g} \leftarrow \frac{\partial E}{\partial W}, \quad \|\hat{g}\| \geq \text{threshold} \Rightarrow \hat{g} \leftarrow \frac{\text{threshold}}{\|\hat{g}\|} \hat{g}.$$

This method is called **gradient clipping**. It preserves the direction of the gradient while limiting its magnitude, preventing excessively large parameter updates.

Vanishing gradients

Common approaches include:

- Initialize $W^{(hh)}$ as the identity matrix rather than randomly.
- Use ReLU instead of sigmoid. The derivative of ReLU is either 0 or 1; where it is 1, the gradient does not continuously decay as it propagates backward through time.

2.5 Deep Bidirectional RNNs

So far, we have focused on RNNs that predict the next word from previous words. By allowing another RNN to read the corpus in reverse, we can also use information from future words. A standard RNN uses only past information, whereas a bidirectional RNN maintains two hidden states:

- $\vec{h}_t$ propagates from left to right and encodes past information.
- $\overleftarrow{h}_t$ propagates from right to left and encodes future information.

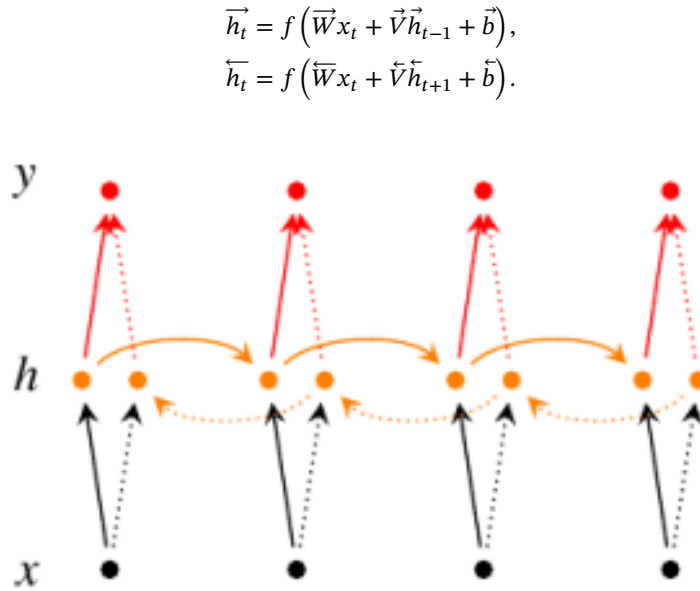


Figure 4: A bidirectional RNN

The hidden states from the two directions are concatenated to make a prediction:

$$\hat{y}_t = g\left(U[\vec{h}_t; \overleftarrow{h}_t] + c\right).$$

A bidirectional RNN can use context on both sides of the current position, but it must maintain two sets of hidden states and parameters.

Deep bidirectional RNNs

RNNs can be stacked into multiple layers. The output of a lower layer becomes the input to the next layer at the same time step. For layer i ,

$$\begin{aligned}\vec{h}_t^{(i)} &= f\left(\vec{W}^{(i)} h_t^{(i-1)} + \vec{V}^{(i)} \vec{h}_{t-1}^{(i)} + \vec{b}^{(i)}\right), \\ \overleftarrow{h}_t^{(i)} &= f\left(\overleftarrow{W}^{(i)} h_t^{(i-1)} + \overleftarrow{V}^{(i)} \overleftarrow{h}_{t+1}^{(i)} + \overleftarrow{b}^{(i)}\right).\end{aligned}$$

The two directional hidden states from the highest layer are used for prediction:

$$\hat{y}_t = g\left(U[\vec{h}_t^{(L)}; \overleftarrow{h}_t^{(L)}] + c\right).$$

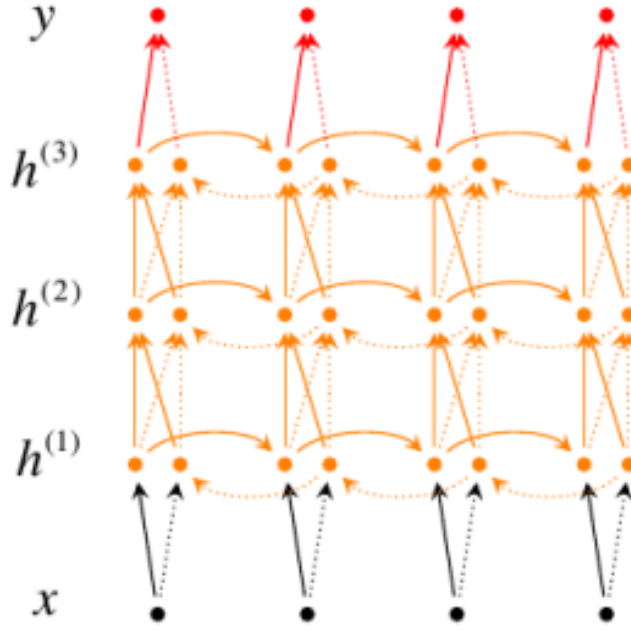


Figure 5: A deep bidirectional RNN

3 Gated Recurrent Units (GRUs)

Beyond these extensions, researchers have found that more complex activation units can improve RNN performance. So far, we have transformed h_{t-1} into h_t using an affine transformation followed by an element-wise nonlinearity. A GRU changes this structure by introducing **gated activation functions**.

Although an RNN can theoretically capture long-term dependencies, it is difficult to train one to do so reliably in practice. The GRU is designed to preserve memory more easily over long periods, making long-range dependencies easier to capture. It generates the next hidden state h_t from h_{t-1} and x_t in four steps:

1. **Update gate:** Determines how much of the previous hidden state h_{t-1} is retained directly in the current state.

$$z_t = \sigma(W^{(z)}x_t + U^{(z)}h_{t-1}).$$

When $z_t \approx 1$, the unit mainly preserves the old memory. When $z_t \approx 0$, it mainly uses the new memory.

2. **Reset gate:** Determines how much past information is used when generating the new memory.

$$r_t = \sigma(W^{(r)}x_t + U^{(r)}h_{t-1}).$$

The smaller r_t is, the more the unit tends to ignore the previous hidden state when generating new memory.

3. **New memory generation:** Combines the current input x_t with past information filtered by the reset gate to produce a candidate hidden state.

$$\tilde{h}_t = \tanh(r_t \circ U h_{t-1} + W x_t).$$

4. **Hidden state:** Uses the update gate to form a weighted combination of the old memory h_{t-1} and the new memory $\tilde{h}_t$.

$$h_t = (1 - z_t) \circ \tilde{h}_t + z_t \circ h_{t-1}.$$

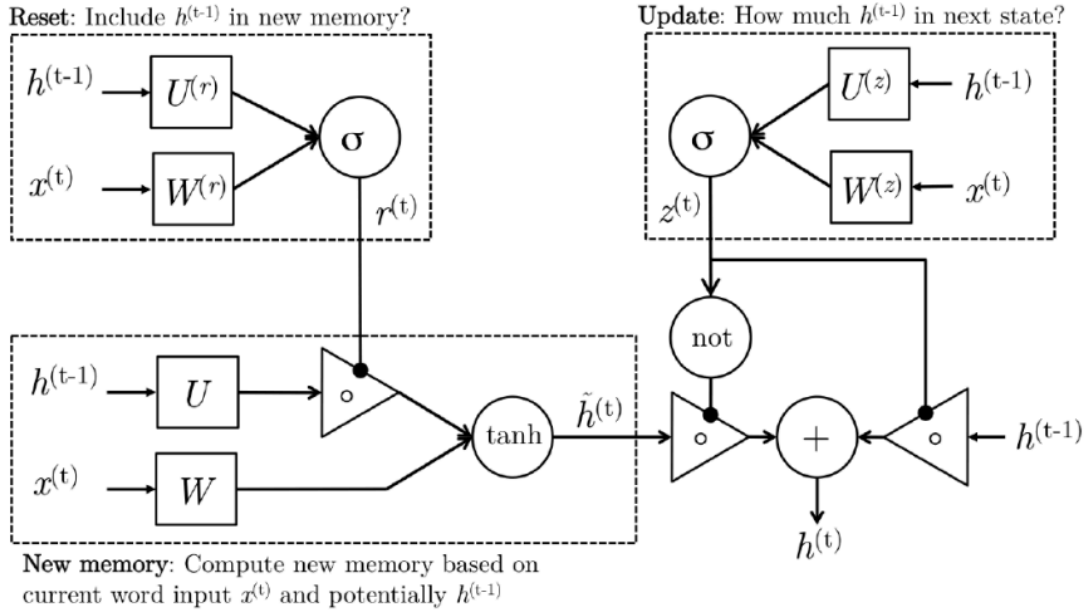


Figure 6: The gated structure of a GRU

4 Long Short-Term Memory Networks (LSTMs)

An LSTM controls information flow with an **input gate**, a **forget gate**, and an **output gate**. It also maintains a memory cell c_t that stores long-term information. The LSTM can be understood in the following stages:

1. **New memory generation:** Generates candidate memory from the current input x_t and the previous hidden state h_{t-1} .

$$\tilde{c}_t = \tanh(W^{(c)}x_t + U^{(c)}h_{t-1}).$$

2. **Input gate:** Determines how much of the candidate memory $\tilde{c}_t$ is written to the current memory.

$$i_t = \sigma(W^{(i)}x_t + U^{(i)}h_{t-1}).$$

3. **Forget gate:** Determines how much information from the previous memory c_{t-1} is retained.

$$f_t = \sigma(W^{(f)}x_t + U^{(f)}h_{t-1}).$$

4. **Final memory generation:** Combines the retained old memory with the newly written memory to produce the current memory c_t .

$$c_t = f_t \circ c_{t-1} + i_t \circ \tilde{c}_t.$$

5. **Output gate and hidden state:** The output gate determines how much of the memory c_t is exposed through the hidden state h_t .

$$o_t = \sigma(W^{(o)}x_t + U^{(o)}h_{t-1}), \quad h_t = o_t \circ \tanh(c_t).$$

In summary:

- i_t controls **how much new information is written**.
- f_t controls **how much old information is retained**.
- o_t controls **how much of the current memory is exposed**.
- c_t is the long-term memory.
- h_t is the hidden state exposed at the current time step.

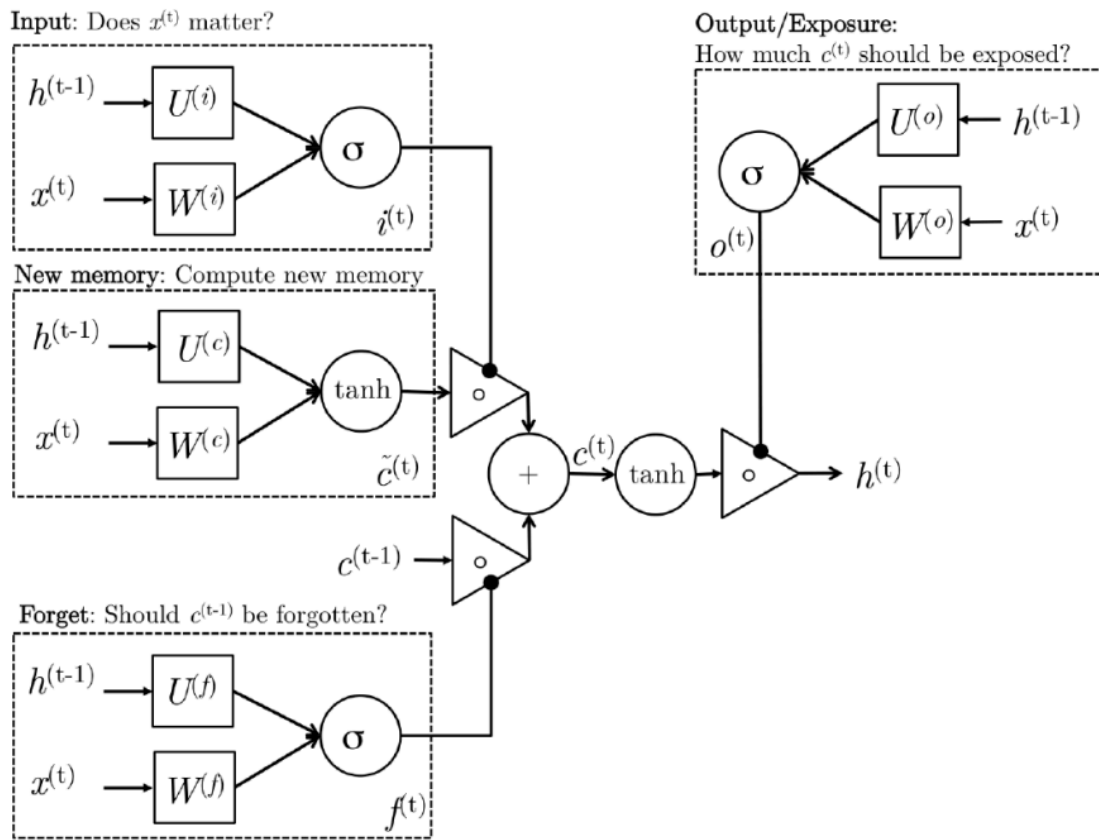


Figure 7: The gated structure of an LSTM